

雲端系統課程 分組專題 (2) 架構分析報告

分散式計算服務系統

商場購物搶購多代理人路徑規劃服務

繳交日期：2026 年 05 月 27 日

組員姓名：陳泓瑜

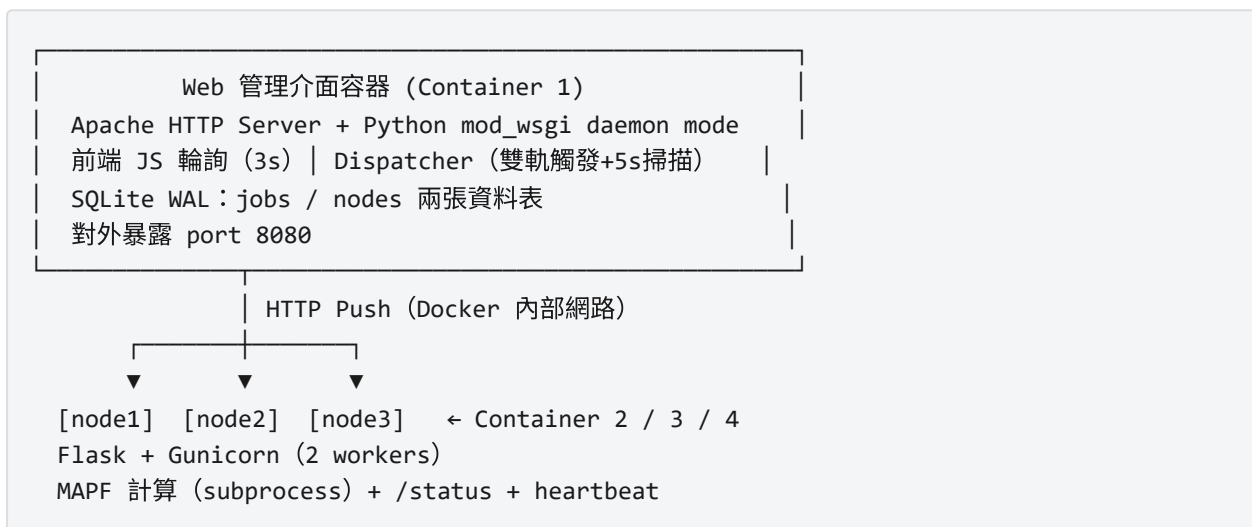
學 號：B1229057

壹、本系統架構說明

1.1 系統概述與目標

本系統為一套以 Docker 容器化部署的分散式計算服務平台，提供商場購物搶購多代理人路徑規劃（Multi-Agent Pathfinding, MAPF）的計算服務。系統由一個 Web 版工作管理介面與三個計算節點共計四個 Docker Container 組成，以 Docker Compose 統一管理網路與啟動順序。使用者可透過 Web 介面設計商場地圖、設定顧客人數與購物清單，提交模擬任務後系統自動排程分派至空間節點，完成後回傳 GIF 動畫呈現搶購過程。

1.2 整體架構與資料流



使用者提交任務 → SQLite 寫入 (status=queued) → Dispatcher HTTP POST 推送節點 → 節點背景計算 → 完成後 POST callback 回傳 GIF → 前端輪詢更新顯示。所有容器以 hostname 互相定址 (`web / node1 / node2 / node3`)。

1.3 容器職責分工

容器	技術	主要職責
Container 1 (Web)	Apache + mod_wsgi + Python + SQLite	工作提交、佇列管理、分派邏輯、監控 介面、資料持久化
Container 2/3/4 (節點)	Python + Flask + Gunicorn	接收任務、執行 MAPF 計算、產出 GIF、回報資源與心跳

貳、技術選型與核心設計決策

2.1 Apache + mod_wsgi Daemon Mode

作業要求採用傳統 Apache HTTP Server。原生 CGI 模式每次請求啟動新程序，無法維護跨請求的狀態與背景執行緒。因此採用 **mod_wsgi Daemon Mode**：Python 以獨立程序池持續運行，Apache 代理請求至 WSGI 程序，WSGI 程序可自行啟動定期分派的 daemon thread，根本解決 CGI 的無狀態限制。

2.2 SQLite + WAL Mode

四容器限制排除 Redis 等額外服務。mod_wsgi 多程序間無共享記憶體，須透過外部儲存維護共享狀態。選用 **SQLite WAL (Write-Ahead Logging) Mode**：WAL 允許多讀一寫並發，適合監控頁面頻繁讀取的存取模式；所有狀態轉換在 `BEGIN IMMEDIATE TRANSACTION` 內完成，確保原子性。UUID v4 作為 Job ID 防止工作總量洩露，並支援幂等提交。

2.3 Push 模式雙軌 Dispatcher

工作分派採 **Push 模式**（管理端主動呼叫節點 API），管理端在分派當下記錄目標節點，直接滿足「標明正在哪個節點」的要求。Dispatcher 採雙軌觸發：**事件觸發**（工作提交 / callback 到達時立即分派）搭配**背景掃描**（每 5 秒執行週期性掃描，補救因 callback 失敗而滯留的工作）。節點每 10 秒發 Heartbeat，35 秒無回應則標記 offline，Circuit Breaker 阻止分派至失效節點。

2.4 計算節點：Flask + Gunicorn (2 Workers)

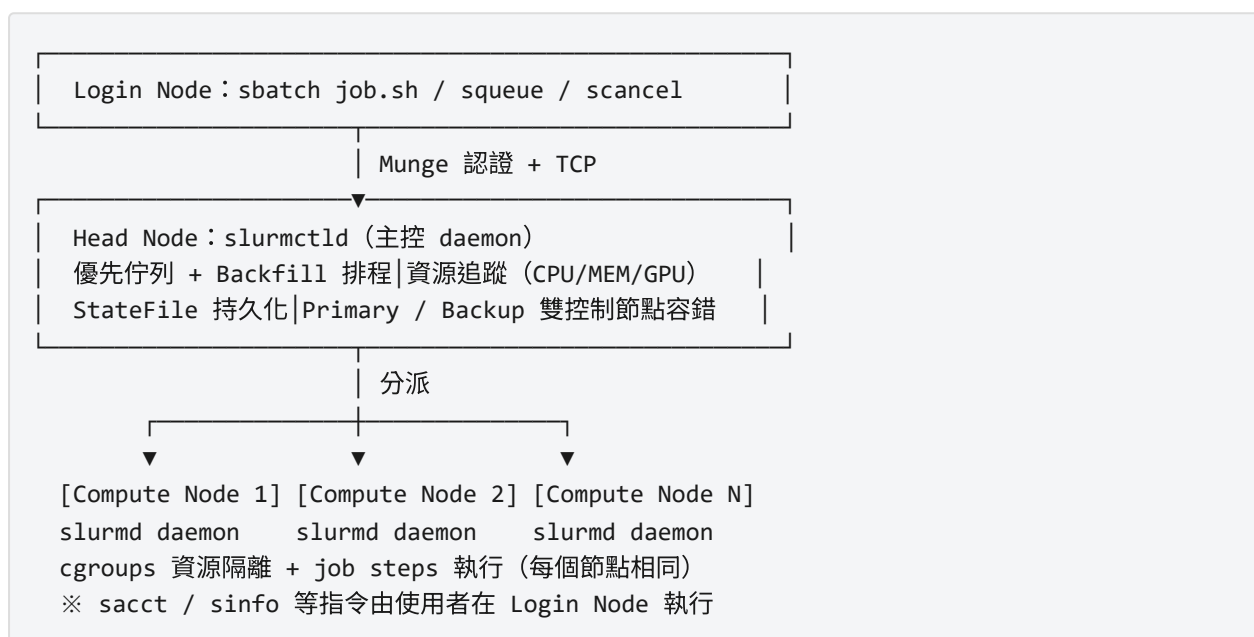
節點以 **Gunicorn 兩 worker** 部署：一個執行長時間 MAPF 計算（subprocess.Popen 獨立程序，規避 Python GIL），一個回應 `/status` 與 Heartbeat，確保計算中節點仍可回報狀態。`/run` endpoint 收到請求立即回傳 202，計算完成後以指數退避（1s→2s→4s）重試 callback。

參、比較對象：SLURM 系統介紹

3.1 系統概述

SLURM (Simple Linux Utility for Resource Management) 是目前全球 HPC 叢集中被廣泛採用的開源工作排程系統，由 Lawrence Livermore National Laboratory 開發，目前已在眾多學術機構與研究單位的高效能運算環境中部署。SLURM 核心功能與本系統高度相似：接受工作提交、維護佇列、分派至節點執行，並提供即時狀態與資源查詢。

3.2 SLURM 核心架構



`slurmctld` 集中管理所有排程決策；各節點的 `slurmd` 執行 job steps 並以 cgroups 在 OS 層面限制資源。使用者以 CLI 指令操作 (`sbatch / queue / scancel / sinfo`)，功能集與本系統 REST API 直接對應。

肆、本系統與 SLURM 比較分析

4.1 架構與部署

兩系統皆採集中式「主從 (Master-Worker)」架構。本系統以 Docker Compose 管理四容器，可在單台機器以單一指令完成部署，設定門檻低；SLURM 部署於多台實體或虛擬機叢集，需設定 `slurm.conf`、Munge、網路路由等，部署程序較為複雜，但支援大規模節點擴展。通訊安全上，SLURM 以 Munge HMAC 認證所有 daemon 訊息；本系統以純 HTTP 於 Docker 內部網路通訊，適合受控環境。

4.2 排程策略與資源管理

本系統採 Round-Robin 輪詢配合「每節點同時最多一工作」策略，邏輯清晰，便於展示佇列行為。SLURM 實作多佇列優先排程 (FIFO / Backfill 插件)，可依 CPU 核心、記憶體量、GPU 數、時間限制細粒度分配資源，並支援工作搶占 (Preemption)。資源監控方面，本系統輪詢節點 `/status` endpoint 解析 `top` 輸出；SLURM 透過 cgroups 精確計帳，完成後可輸出詳細資源使用報表 (`sacct`)。

4.3 容錯與高可用性

本系統以 Heartbeat + Circuit Breaker 偵測節點失效，失效節點工作透過背景掃描重排 (最多重試 3 次)。然而 Web 容器本身為單點失效點 (SPOF)，容器故障則服務中斷。SLURM 的 Primary/Backup `slurmctld` 架構中，Backup 節點可在短時間內完成控制權移交，並從 StateFile 完整恢復工作狀態，降低了管理端的單點風險。

4.4 綜合比較表

比較維度	本系統	SLURM
架構模式	集中式 manager + HTTP push	集中式 slurmctld + slurmd daemon
部署方式	Docker Compose (單機可執行)	實體 / 虛擬機叢集
狀態持久化	SQLite WAL (volume 掛載)	StateFile + checkpoint
排程策略	Round-Robin · 1 job / node	多佇列 FIFO / Backfill
資源分配粒度	節點層級	CPU 核心 / 記憶體 MB / GPU
節點健康偵測	Heartbeat (10s) + Circuit Breaker	slurmd 心跳 + NodeHealthCheck
管理端容錯	無 (SPOF)	Primary / Backup slurmctld
使用者介面	Web UI + REST API	CLI (sbatch/squeue) + REST API
通訊安全	純 HTTP (Docker 內部)	Munge 認證 + 可配置 TLS
擴展性	手動修改 Compose 檔	動態加入節點 · 支援大規模叢集
部署複雜度	低 (單一指令啟動)	高 (需 OS 層設定)
適用規模	小型 (<10 節點 · 教學)	大型 HPC 叢集

伍、本系統優缺點分析

5.1 優點

優點	說明
容器化部署，環境可重現	以 docker compose up 單一指令啟動全系統，依賴封裝於映像檔，消除環境差異問題
工作狀態視覺化	Web 介面清晰呈現三種工作狀態，執行中工作標示所在節點，完成工作提供 GIF 動畫預覽
業界標準設計模式	雙軌 Dispatcher、Heartbeat + Circuit Breaker、指數退避重試等模式與生產系統設計一致，具教育意義
計算任務具真實應用背景	MAPF 對應 Amazon Robotics 等倉儲機器人場景，計算量充分（15–35 秒），輸出 GIF 動畫易於呈現結果

5.2 缺點與限制

缺點	說明	改善方向
管理節點 SPOF	Web 容器故障則服務全停，狀態有遺失風險	引入資料庫主從複寫或 Leader Election
排程缺乏資源感知	Round-Robin 不考慮節點當前負載，可能分派至高負載節點	查詢 /status，採 Least-Loaded First
內部通訊未加密	純 HTTP 無身份驗證，不適合對外開放的生產環境	引入 JWT token 或 mTLS
前端輪詢有延遲	3 秒輪詢間隔導致狀態更新最多延遲 3 秒，產生固定背景流量	改用 WebSocket 或 SSE 即時推送

陸、結論

本系統與 SLURM 在核心功能定位上高度一致：工作提交、佇列排程、節點分派與狀態監控均有對應實作。主要差距在於系統規模與容錯能力——本系統以最低基礎設施成本實現分散式計算服務的核心概念，SLURM 則以多年工程積累與持續演進應對大規模 HPC 的生產需求。

就適用情境而言：

- **本系統適合：**教學環境中模擬分散式排程行為，以及節點數少於 10 個、不需高可用性的小型計算服務

- **SLURM 適合**：大學 / 研究機構的 HPC 資源共享平台，以及任何需要細粒度資源管理、多用戶公平排程與高可用性保障的生產環境

本專題透過設計並實作這套系統，深入實踐了工作佇列、節點健康偵測、多程序並發與容器化部署等分散式系統核心概念。與 SLURM 的比較揭示了從教學原型到生產系統的工程距離，亦為後續系統強化提供了清晰的改進方向。